Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 2: Computer Code and Data Storage

Aurélien Sallin (lecturer) and Matthias Roesti (class tutor)

05/10/2023

Prerequisites

- A text editor of some sort. Install the latest version of the text editor Sublime Text here: https://www.sublimetext.com/download. Another alternative is Atom: https://atom.io/.

Exercises

Exercise A: Loading text files in RStudio

When working with data, a first important step is to load your data, in our case in RStudio. As you know, data can come in different formats.

1. Along with the exercise instructions, you will find two datasets on Canvas, `simple_data.txt` and `simple_data.csv`. Download them and save/upload them into the `r_course/data` folder (create data subfolder if not already existing). Further ensure that your working directory is the `r_course` folder.

2. Check out the files `simple_data.txt` and `simple_data.csv` by opening them (outside of RStudio) in a text editor, such as Sublime Text or Atom. Do they look different from each other?

3. Go back to RStudio and load the data in .txt format as follows:

```r
df1 <- read.table("data/simple_data.txt", header = FALSE)
```

Type `df1` or `print(df1)` into the console. What do you see?

4. Next, turn to the file in .csv format. Run the following code to load it:

```r
df2 <- read.table("data/simple_data.csv", header = FALSE, sep=",")
```

Again, type `df2` or `print(df2)` into the console. Now, what do you see? Compare this output to the output from `print(df1)`.

5. Generate a text file containing the value in the third column of the second row of `df1`. Hint: you can obtain values from dataframes by indexing like this: `df[i,j]`, where i and j indicate row and column numbers, respectively. Save the text file as "df1_value.txt" in your `r_course/data` folder.

Exercise B: Downloading and importing data on air quality

In the following, we are going to download and import data on air quality in New York.

1. Using a browser of your choice, navigate to this page by the US data.gov data catalog: https://catalog.data.gov/dataset/air-quality.

2. Scroll down to the Section 'Downloads & Resources' and download the data file in .csv format and put it into your `r_course/data` folder.

3. Based on Exercise A, Part 4, how would you open this dataset in RStudio? Hint: In contrast to the files in Exercise A, this file comes with a header row. Complete the following code accordingly:

```
df_airquality <- read.table(...)
```

4. Once you have loaded the data into RStudio, you can inspect the first (or last) couple of rows using `head(df_airquality)` (or `tail(df_airquality)`).
5. Create a text file with the amount (in variable `Data.Value`) for fine particulate matter pollution (variable `Name == "Fine Particulate Matter (PM2.5)"`) in the winter of 2008/09 (variable `Time.Period == "Winter 2008-09"`) in lower Manhattan (variable `Geo.Place.Name == "Lower Manhattan"`). Save your text file as "airquality_manhattan.txt" in your `r_course/data` folder. Hint: use the `subset()` command, and note that you can link different subsetting conditions with the `&` operator.

Exercise C: Exploring computer code - HTML - let's manipulate the media!

In this exercise, we will explore HTML code, which is the basis of most websites.

1. Open your preferred web browser (e.g. Firefox, Chrome or Safari. Note that Internet Explorer is never a preferred browser). Go to the following website: `https://www.foxnews.com/`.
2. Now let's look at the HTML source code for this website - see *here* for information on how to do this in your browser of choice. Try to recognize parts of the code that correspond to elements of the displayed page.
3. Pick a news headline on the site and find it in the source code - on Windows, you can use `CTRL + F` to use the search bar to look for it in the source code by copying the first couple of words from the headline and putting it into the search bar. Note: most browsers also have an 'inspect element' option when you right click on/select the element you want to look at. This can take you directly to the corresponding part of the HTML (or at least to the vicinity, sometimes you also have to try a couple of times to get where you want to be). For more information on how to do this (in Safari, you will need to activate developer mode for example), see the following *link*

For example, I found the following segment in the HTML code relating to an article titled 'Republicans sound alarm over CCP-linked businessman's cash offer to Hunter Biden' (no subtle political messaging intended):

```
<!DOCTYPE html>
<html>

...

  <article class="article story-2">
      <div class="m">
          <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy-
relationship-ccp-linked-businessman-damning-evidence" data-omtr-intcmp="fnhpbt3" data-adv=
  "hp1bt">

            <picture>
                <source media="(max-width: 767px)" srcset="//a57.foxnews.com/prod-hp.fox
news.com/images/2023/10/343/193/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/686/386/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 768px) and (max-width: 1023px)" srcset="//a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/335/188/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/670/376/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
```

```
                <source media="(min-width: 1024px) and (max-width: 1279px)" srcset="//a57.
                foxnews.com/prod-hp.fox
news.com/images/2023/10/303/170/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57
.foxnews.com/prod-hp.foxnews.com/images/2023/10/606/340/57
6c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 1280px) and (max-width: 1439px)" srcset="//a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/277/156/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1, //a57.foxnews.com/prod-hp.foxnews.com/images/2023/10/554/312/576c017a6f
05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1 2x">
                <source media="(min-width: 1440px)" srcset="//a57.foxnews.com/prod-hp.fox
news.com/images/2023/10/331/186/576c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1, //a57.
foxnews.com/prod-hp.foxnews.com/images/2023/10/662/372/576c017a6f05588ac891d4aa76dbd0e7.
jpg?tl=1&amp;ve=1 2x">
                <img src="//a57.foxnews.com/prod-hp.foxnews.com/images/2023/10/343/193/576
c017a6f05588ac891d4aa76dbd0e7.jpg?tl=1&amp;ve=1" width="343" height="193" alt="Republicans
sound alarm over CCP-linked businessman's cash offer to Hunter Biden">
            </picture>
            <div class="kicker default"><span class="kicker-text">'IN THEIR POCKET'</span>
            </div>
          </a>
          <span class="pill is-breaking">EXCLUSIVE</span>
      </div>

      <div class="info ">
        <header class="info-header">
          <h3 class="title ">
            <a href="https://www.foxnews.com/politics/gop-lawmakers-rip-hunter-bidens-cozy
            -relationship-ccp-linked-businessman-damning-evidence#&amp;_intcmp=fnhpbt3"
            data-omtr-intcmp="fnhpbt3">Republicans sound alarm over CCP-linked business-
            man's cash offer to Hunter Biden</a>
          </h3>
        </header>
        <div class="content">


        </div>
      </div>

  </article>

...

</html>
```

4. Now it's time for some good old media manipulation. In the HTML-code, replace the chosen news headline with a title of your choice (if you need some inspiration, see *here*). In most browsers, you can double click the source code to edit it, then confirm your edits by hitting the enter/return key. What can you see? Did the website change?

5. BONUS (not exam relevant): Can you figure out a way to replace the article teaser image as well? Hint: this is easiest if you use online image addresses, e.g. https://static.wikia.nocookie.net/mrbean/images/4/4b/Mr__beans__holiday__ver2.jpg or https://preview.redd.it/bad-luck-brian-meme-hd-upscaled-v0-syiviojoq72a1.jpg?width=1080&crop=smart&auto=webp&s=eace4bc6be838bfe3f30219743d5a5c34c5f4335)

Exercise D: Downloading and importing data on air quality (XML format)

1. I provided you with a data file `Air_Quality.xml` on Canvas. Save it in your `r_course/data` folder.
2. Load the data in .xml format. Hint: The package `XML` might be of use. You can install new packages as follows: `install.packages("package_name")`.
3. Once you have loaded the .xml file in Part 2, how can you convert it to an R dataframe?

Exercise E: Read raw text data

This exercise guides you through how we can import raw text data into R. Suppose you are writing your BA Thesis on the topic "The Effect of Climate Change on Consumption Behavior in Switzerland". For your empirical analysis you need, inter alia, standardized data on temperature anomalies in Switzerland (as a proxy for the salience of climate change). You identify NASA's Earth System Data Explorer as a promising data source and aim to download and work with this data in R. The exercise guide's you exemplary though how this can be done.

Hint: This exercise is an adapted version of the tutorial in Chapter 1 of Paul Murell's book (see syllabus). Read the chapter for an additional introduction to data stored in text files.

1. Go to NASA's Earth System Data Explorer: https://mynasadata.larc.nasa.gov/EarthSystemLAS/UI.vm.
2. Click on *Data Set* in the upper left corner and navigate through `Atmosphere/Featured Phenomena/Changing Air Temperatures/Temperatures` and select `Monthly Surface Air Temperature Anomaly`.
3. On the bottom left part of the page, under `Line Plots`, select `Time`.
4. Enter the coordinates 47.25 N and 9.22 W (St. Gallen) and click `Update Plot` in the upper left corner. The page shows a plot of the monthly surface air temperature anomaly time series.
5. Click on `Download Data ...`. In the new tab select `ASCII` as a data format. Click to save.

The downloaded ASCII-file containing the raw data should open in a new tab. Inspect the data and save (`Save page as...`) it as `temp_anomaly.txt` in your `r_course/data` folder.

Now we can read the data into R:

```r
# install the readr package if not yet installed: install.packages("readr")
library(readr)
# read the data
ta <- read_tsv("data/temp_anomaly.txt", skip = 9, col_names = c("month", "temp_anomaly"))
```

```r
# inspect the first rows
head(ta)
```

- What is the purpose of the `skip` argument and why is it set to `9`?
- Why do we use `read_tsv()` in this situation and not `read_csv()`?